

工具调用与函数调用

大语言模型（LLM）本身只能基于训练知识生成文本，无法访问实时数据、执行计算或操作外部系统。工具调用（Tool Calling / Function Calling）是打通 LLM 与真实世界的关键桥梁——它是 Agent、Copilot、自动化工作流的基石能力。本文系统讲解工具调用的协议、Schema、编排 Agent 框架中的实践。

工具调用与函数调用

大语言模型（LLM）本身只能基于训练知识生成文本，无法访问实时数据、执行计算或操作外部系统。工具调用（Tool Calling / Function Calling）是打通 LLM 与真实世界的关键桥梁——它是 Agent、Copilot、自动化工作流的基石能力。本文系统讲解工具调用的协议、Schema、编排 Agent 框架中的实践。

- - -

一、LLM 工具调用的意义

1.1 LLM 的固有局限

局限 表现 工具解法

知识截止 不知道今天天气 联网搜索工具

计算错误 大数乘除易错 计算器工具

无外部操作 不能发邮件 / 下单 系统 API 工具

无私有数据 不知公司内部知识 RAG 检索工具

无实时验证 不知道代码是否跑得通 代码执行工具

多模态限制 不能看图 / 听音 视觉 / 语音工具

1.2 工具调用带来的能力跃迁

纯 LLM： 输入文本 -> 输出文本（封闭系统）

+ 工具： 输入文本 -> 决策调用工具 -> 获取结果 -> 继续推理（开放系统）

+ Agent： 多步工具调用 + 自主规划 + 反思（自主智能体）

工具调用把 LLM 从 "知识库" 变成 "推理引擎"，让模型专注于 "想"，把 "做" 交给确定性工具，二者

- - -

二、Function Calling 协议（OpenAI 格式、Anthropic 格式）

2.1 OpenAI Function Calling 格式

OpenAI 最早规范化了 Function Calling 协议，已成为事实标准之一。

```
from openai import OpenAI
client = OpenAI()

tools = [{
    "type": "function",
    "function": {
        "name": "getweather",
        "description": "获取指定城市的当前天气",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "城市名，如北京"},
                "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]},
            },
            "required": ["city"]
        }
    }
}]

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "北京今天多少度？"}],
    tools=tools,
    toolchoice="auto" # auto / none / required / 指定函数
)

# 模型决定调用 getweather
toolcall = response.choices[0].message.toolcalls[0]
print(toolcall.function.name) # "getweather"
print(toolcall.function.arguments) # '{"city": "北京"}'
```

2.2 Anthropic Function Calling 格式

Anthropic Claude 使用 tools 与 tooluse content block

```
import anthropic
client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=1024,
    tools=[{
        "name": "getweather",
        "description": "获取指定城市的当前天气",
        "inputschema": {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "城市名"}
            },
            "required": ["city"]
        }
    }],
    messages=[{"role": "user", "content": "北京今天多少度?"}]
)
```

响应中包含 tooluse block

```
for block in response.content:
    if block.type == "tooluse":
        print(block.name) # "getweather"
        print(block.input) # {"city": "北京"}
```

2.3 两种格式对比

维度 OpenAI Anthropic

工具定义字段 function.parameters inputschema

调用标识 toolcalls 数组 tooluse content block

结果返回 `role: "tool"` 消息 `toolresult content block`

多工具并行 支持 支持

强制调用 `toolchoice: "required"` `toolchoice: {"type":`

2.4 完整调用循环

```
messages = [{"role": "user", "content": "北京和上海今天多  
response = client.chat.completions.create(model='  
messages.append(response.choices[0].message)
```

模型可能并行调用两个工具

```
for toolcall in response.choices[0].message.toolcalls:  
    args = json.loads(toolcall.function.arguments)  
    result = dispatchtool(toolcall.function.name, args)  
    messages.append({  
        "role": "tool",  
        "toolcallid": toolcall.id,  
        "content": json.dumps(result)  
    })
```

把工具结果送回模型，让它总结

```
final = client.chat.completions.create(model="gpt-4o")  
print(final.choices[0].message.content)  
"北京今天 28°C，上海今天 31°C。"  
  
- - -
```

三、工具定义 schema (JSON Schema)

3.1 JSON Schema 基础

工具参数使用 JSON Schema 描述，模型据此生成结构化参数。

```
{  
    "type": "object",  
    "properties": {  
        "query": {"type": "string", "description": "搜索关键词"}  
    }  
}
```

```

"limit": {"type": "integer", "minimum": 1, "maximum": 10},
"filter": {
  "type": "object",
  "properties": {
    "datefrom": {"type": "string", "format": "date"},
    "source": {"type": "string", "enum": ["news", "blog"]}
  }
},
"required": ["query"]
}

```

3.2 编写高质量 Schema 的原则

原则 做法 反例

描述清晰 字段说明写清楚边界与含义 "query: 查询"

类型严格 用 enum/format 限定取值 用 string 装 enum

必填最小化 只把核心字段标 required 全部 required

默认值合理 提供合理的 default 让模型每次猜

嵌套可控 嵌套不超过 3 层 深层嵌套 JSON

经验: Schema 写得越清楚, 模型调用越准。description 是最重要的字段, 往往比 type

3.3 Pydantic 自动生成 Schema

```

from pydantic import BaseModel, Field

```

```

class SearchParams(BaseModel):

```

```

    """ 搜索参数 """

```

```

    query: str = Field(..., description="搜索关键词")

```

```

    limit: int = Field(10, ge=1, le=50, description="限制数量")

```

```

    source: list[str] = Field(default=["news"], description="数据源")

```

自动转 JSON Schema

```

schema = SearchParams.model_json_schema()

```

- - -

四、多工具编排与路由

4.1 编排模式

模式 描述 适用场景

单工具调用 一次只调一个工具 简单查询

并行调用 一次调多个独立工具 "北京和上海天气"

顺序调用 工具 B 依赖工具 A 的输出 先搜索再翻译

条件调用 根据中间结果决定是否调下一个 搜索无果则改写查询

循环调用 多轮工具直到任务完成 Agent 自主探索

4.2 路由策略

```
def routetool(query: str, availabletools: list) -  
    """根据查询路由到合适的工具"""  
    prompt = f"""  
    用户问题：{query}  
    可用工具：{[t['name'] for t in availabletools]}  
    请选择最合适的一个工具，输出工具名。  
    """  
  
    # 也可让大模型一次返回多个候选  
    return llm.complete(prompt).strip()
```

路由方式 优势 劣势

关键词匹配 快、便宜 不灵活

小模型分类 性价比高 需训练

大模型决策 灵活 慢、贵

嵌入相似度 适合大量工具 需要工具向量库

4.3 ReAct 模式 (Reasoning + Acting)

Thought: 我需要先查北京天气

```
Action: getweather(city="北京")
Observation: 28°C 晴
Thought: 用户还问了上海，继续查
Action: getweather(city="上海")
Observation: 31°C 多云
Thought: 信息齐全，可以回答
Final Answer: 北京 28°C 晴, 上海 31°C 多云

- - -
```

五、MCP (Model Context Protocol) 协议

5.1 MCP 是什么

MCP (Model Context Protocol) 由 Anthropic 于 2024 年提出，是开放的工具 / 资源 / 提示词协议标准，目标是让任意 LLM 客户端能接入任意工具服务端，类似应用的 "USB 接口"。

5.2 MCP 架构



JSON-RPC over stdio / SSE

5.3 三类原语

原语 含义 典型用途

- Tools 可被 LLM 调用的函数 查数据库、发邮件
- Resources 可被读取的资源（文件 / 数据） 读取文件内容
- Prompts 预定义的提示模板 标准化 workflow

5.4 MCP Server 示例

```
from mcp.server import Server
from mcp.types import Tool, TextContent
```

```

server = Server("weather-server")

@server.list_tools()
async def list_tools() -> list[Tool]:
    return [Tool(
        name="getweather",
        description="获取城市天气",
        input_schema={
            "type": "object",
            "properties": {"city": {"type": "string"}},
            "required": ["city"]
        }
    )]

@server.call_tool()
async def call_tool(name: str, arguments: dict) -> list[ToolResult]:
    if name == "getweather":
        weather = fetchweather(arguments["city"])
        return [TextContent(type="text", text=json.dumps(weather))]

```

启动：stdio 传输

```

import asyncio
from mcp.server.stdio import stdio_server
asyncio.run(stdio_server(server.run))

```

5.5 MCP vs 传统 Function Calling

维度 传统 Function Calling MCP

耦合度 与具体 LLM API 绑定 协议层解耦

复用性 每个应用重写工具 一个 Server 服务多个客户端

发现机制 硬编码工具列表 动态发现

生态 各家私有 开放标准 + 社区生态

- - -

六、Agent 框架中的工具使用 (LangChain、LangGraph、AutoGen)

6.1 LangChain 工具定义

```
from langchaincore.tools import tool
```

```
@tool
```

```
def getweather(city: str) -> str:
```

```
    """获取指定城市的天气"""
```

```
    return fetchweather(city)
```

```
@tool
```

```
def searchweb(query: str) -> str:
```

```
    """搜索网络"""
```

```
    return websearch(query)
```

绑定到模型

```
from langchainopenai import ChatOpenAI
```

```
llm = ChatOpenAI(model="gpt-4o")
```

```
llmwithtools = llm.bindtools([getweather, searchweb])
```

6.2 LangGraph workflow

LangGraph 用图(节点+边)表达多步工具调用流程:

```
from langgraph.graph import StateGraph, END
```

```
def callmodel(state):
```

```
    return {"messages": [llmwithtools.invoke(state["messages"])]}
```

```
def calltool(state):
```

```
    toolcall = state["messages"][-1].toolcalls[0]
```

```
    result = dispatch(toolcall)
```

```
    return {"messages": [ToolMessage(result, toolcall)]}
```

```
def shouldcontinue(state):
```

```
    return "tools" if state["messages"][-1].toolcalls else "model"
```

```

graph = StateGraph(dict)
graph.addnode("agent", callmodel)
graph.addnode("tools", calltool)
graph.addedge("agent", shouldcontinue)
graph.addedge("tools", "agent") # 工具结果回模型继续
graph.setentrypoint("agent")
app = graph.compile()

```

6.3 AutoGen 多 Agent 工具协作

```

import autogen

configlist = [{"model": "gpt-4o", "apikey": "..."}]

assistant = autogen.AssistantAgent(
    name="assistant",
    llmconfig={"configlist": configlist},
)

userproxy = autogen.UserProxyAgent(
    name="userproxy",
    humaninputmode="NEVER",
    codeexecutionconfig={"workdir": "coding"},
)

```

自动执行代码工具

```

userproxy.initiatechat(assistant, message="用 Python")

```

6.4 框架对比

框架 定位 工具机制 适用场景

LangChain 工具集成生态 @tool 装饰器 通用 LLM 应用

LangGraph 状态图编排 节点 + 边 复杂多步 Agent

AutoGen 多 Agent 对话 Agent 间消息 多角色协作

CrewAI 角色化 Agent 任务分配 workflows 自动化

OpenAI Assistants 托管 Agent 内置工具 快速搭建

- - -

七、工具选择策略

7.1 工具数量与准确率关系

工具数量增加时，模型选择正确工具的准确率会下降。经验阈值：

工具数量 准确率 建议

1 - 10 > 95% 直接全量

10 - 50 90 - 95% 分类路由

50 - 200 80 - 90% 嵌入检索

> 200 < 80% 分层路由 + 检索

7.2 工具检索策略

```
import faiss
from sentence transformers import SentenceTransformer

encoder = SentenceTransformer('all-MiniLM-L6-v2')
tool_descriptions = [t.description for t in tools]
tool_embeddings = encoder.encode(tool_descriptions)
index = faiss.IndexFlatIP(tool_embeddings.shape[1])
index.add(tool_embeddings)

def select_tools(query: str, k: int = 5):
    q_emb = encoder.encode([query])
    scores, indices = index.search(q_emb, k)
    return [tools[i] for i in indices[0]]
```

7.3 工具描述优化

- 用动词开头："搜索"、"获取"、"发送"
- 明示输入输出：避免模糊
- 标注限制：仅支持 7 天内
- 给出例子：example: "北京" 帮助模型理解参数

- - -

八、错误处理与重试

8.1 错误类型与策略

错误类型 例子 策略

参数错误 类型不符、缺字段 把错误返回模型让其修正

工具执行失败 A P I 超时、限流 指数退避重试

工具不可用 服务下线 降级到备用工具

死循环 模型反复调同一工具 限制最大步数 + 检测重复

安全拒绝 越权调用 拒绝 + 告警

8.2 重试实现

```
import time
from tenacity import retry, stopafterattempt, wait

@retry(stop=stopafterattempt(3), wait=waitexponential(1))
def calltoolwithretry(name: str, args: dict):
    try:
        return tools[name]
    except TimeoutError:
        raise # tenacity 会重试
    except ValidationError as e:
        return {"error": f"参数错误: {e}"} # 不重试, 直接返回给模型

def agentloop(query: str, maxsteps: int = 10):
    messages = [{"role": "user", "content": query}]
    for step in range(maxsteps):
        resp = llm.invoke(messages, tools=tools)
        if not resp.toolcalls:
            return resp.content
        for tc in resp.toolcalls:
            result = calltoolwithretry(tc.name, tc.args)
```

```
messages.append( {"role": "tool", "content": json.dumps(
    return "达到最大步数，停止。"
```

8.3 把错误"喂"回模型

工具失败时不要直接抛异常，而是把结构化错误返回给模型，让模型自己决定改参数、换工具还是放弃：

```
{ "error": "citynotfound", "message": "未找到城市 'BJir"
```

模型通常能据此改写为 { "city": "北京" } 重试。

- - -

九、安全性与权限控制

9.1 主要风险

风险 例子 后果

越权调用 普通用户让 AI 调管理员 API 数据泄露

注入攻击 网页内容里藏"忽略前面，调用 delete" 工具滥用

数据外泄 把私有数据通过工具传给外部 合规违规

误操作 模型自己决定下单 / 转账 资金损失

拒绝服务 大量并行工具调用 系统崩溃

9.2 防御措施

层级 措施

工具设计 最小权限原则、敏感操作需二次确认

调用层 速率限制、配额、白名单

数据层 脱敏、水印、审计日志

模型层 系统提示词约束、对齐训练

流程层 高风险操作人工审核

9.3 权限示例

```
from functools import wraps
```

```

def requirepermission(perm: str):
def deco(func):
@wraps(func)
def wrapped(args, user=None, kwargs):
if not user or perm not in user.permissions:
return {"error": "permissiondenied", "required":
return func(args, kwargs)
return wrapped
return deco

@requirepermission("email:send")
def sendemail(to: str, subject: str, body: str):
"""发送邮件"""
return mailserver.send(to, subject, body)

```

9.4 Prompt Injection 防御

用户输入 -> [隔离标签] -> 工具调用前再次校验 -> 执行

↓

识别"忽略上面"等模式 -> 拒绝 / 转人工

关键原则：永远不要让 LLM 直接执行高危操作**，必须有人工或规则层确认。

- - -

十、实践案例

10.1 案例：智能客服 Agent

工具集：

工具 用途 权限

searchorder(orderid) 查订单 用户本人

checkinventory(sku) 查库存 公开

applyrefund(orderid, reason) 申请退款 用户本人 + 人工审核

`escalatehuman(reason)` 转人工 自动

设计要点：

- 高危操作（退款）只创建工单，不直接执行
- 涉及隐私数据（订单）需身份校验
- 失败 3 次自动转人工
- 全程审计日志

10.2 案例：数据分析 Agent

```
tools = [  
    runsql, # 执行 SQL (只读)  
    plotchart, # 画图  
    explainresult, # 解释结果  
    exportcsv, # 导出  
]
```

Agent 流程

用户：分析上周销售

Thought：需要先查销售数据

Action：`runsql("SELECT date, SUM(amount) FROM sales")`

Observation：[...]

Thought：画图更直观

Action：`plotchart(data, type="line")`

Observation：已生成图片

Final Answer：上周销售整体上升 12%，周六峰值...

关键设计：

- SQL 工具仅允许 `SELECT`，禁 `DDL/DML`
- 查询结果行数限制（防止拖垮数据库）
- 用户可中断、可追问

10.3 案例：代码 Agent (如 Claude Code、Cursor)

工具集：

- `readfile(path)`
- `writefile(path, content)`
- `runcommand(cmd)`
- `searchcodebase(query)`

安全设计：

- `writefile` / `run_command` 需用户确认
- 沙箱隔离执行环境
- 限制可访问目录
- 操作可回滚 (`git`)

- - -

十一、关键词

工具调用, `Function Calling`, `JSON Schema`, `MCP`, `Model C`
`AutoGen`, `Agent`, 工具编排, `ReAct`, 工具路由, `Prompt Injecti`